

UNIVERSIDADE FEDERAL DE GOIÁS
INSTITUTO DE INFORMÁTICA

GABRIEL FERREIRA SILVA

**Inteligência Artificial Generativa e
Engenharia de Software: Um estudo de
caso**

Goiânia
2026

UNIVERSIDADE FEDERAL DE GOIÁS
INSTITUTO DE INFORMÁTICA

**AUTORIZAÇÃO PARA PUBLICAÇÃO DE TRABALHO DE
CONCLUSÃO DE CURSO EM FORMATO ELETRÔNICO**

Na qualidade de titular dos direitos de autor, **AUTORIZO** o Instituto de Informática da Universidade Federal de Goiás – UFG a reproduzir, inclusive em outro formato ou mídia e através de armazenamento permanente ou temporário, bem como a publicar na rede mundial de computadores (*Internet*) e na biblioteca virtual da UFG, entendendo-se os termos “reproduzir” e “publicar” conforme definições dos incisos VI e I, respectivamente, do artigo 5º da Lei nº 9610/98 de 10/02/1998, a obra abaixo especificada, sem que me seja devido pagamento a título de direitos autorais, desde que a reprodução e/ou publicação tenham a finalidade exclusiva de uso por quem a consulta, e a título de divulgação da produção acadêmica gerada pela Universidade, a partir desta data.

Título: Inteligência Artificial Generativa e Engenharia de Software: Um estudo de caso

Autor(a): Gabriel Ferreira Silva

Goiânia, 30 de Junho de 2026.

Gabriel Ferreira Silva – Autor

Jacson Rodrigues Barbosa – Orientador

GABRIEL FERREIRA SILVA

Inteligência Artificial Generativa e Engenharia de Software: Um estudo de caso

Trabalho de Conclusão apresentado à Coordenação do Curso de Computação do Instituto de Informática da Universidade Federal de Goiás, como requisito parcial para obtenção do título de Bacharel em Computação.

Área de concentração: Otimização.

Orientador: Prof. Jacson Rodrigues Barbosa

Goiânia
2026

GABRIEL FERREIRA SILVA

Inteligência Artificial Generativa e Engenharia de Software: Um estudo de caso

Trabalho de Conclusão apresentado à Coordenação do Curso de Computação do Instituto de Informática da Universidade Federal de Goiás como requisito parcial para obtenção do título de Bacharel em Computação, aprovada em 30 de Junho de 2026, pela Banca Examinadora constituída pelos professores:

Prof. Jacson Rodrigues Barbosa
Instituto de Informática – UFG
Presidente da Banca

Prof. William Divino Ferreira
Instituto de Informática – UFG

Paulo Marcos Soares Rodrigues
Gestor de TI – LIGO

Todos os direitos reservados. É proibida a reprodução total ou parcial do trabalho sem autorização da universidade, do autor e do orientador(a).

Gabriel Ferreira Silva

Gabriel Ferreira Silva é graduando em Ciência da Computação pelo Instituto de Informática da Universidade Federal de Goiás (UFG). Durante a graduação, atuou na empresa júnior do Instituto de Informática e estagiou como desenvolvedor *full stack* na Valori, onde atualmente trabalha como desenvolvedor júnior. Tem interesse em desenvolvimento *back-end*, inteligência artificial, banco de dados e arquitetura de software.

Dedico este trabalho à minha mãe.

Agradecimentos

Agradeço à minha mãe, que fez tudo o possível e o impossível para tornar isso realidade;

À minha irmã, que sempre esteve ao meu lado me motivando, até mesmo sem imaginar;

Ao meu orientador, Prof. Jacson Rodrigues Barbosa, pela orientação e pelas oportunidades únicas que me proporcionou;

A todos da Valori, que foram muito importantes no meu aprendizado e sempre me deram todo o suporte necessário;

Ao Laboratório de Inovação Goiás (LIGO), em especial à Gerência de Inovação, Ciência de Dados e Inteligência Artificial, por fornecer o documento de requisitos oficial e as apresentações do estudo de caso real que viabilizaram a validação deste trabalho;

E o meu muito obrigado a todas as pessoas que, em algum momento, fizeram parte dessa jornada, tornando-a muito mais fácil e prazerosa.

Sem vocês, nada disso seria possível.

"The single biggest problem in communication is the illusion that it has taken place."

George Bernard Shaw,

.

Resumo

Ferreira Silva, Gabriel. **Inteligência Artificial Generativa e Engenharia de Software: Um estudo de caso**. Goiânia, 2026. 54p. Relatório de Graduação. Instituto de Informática, Universidade Federal de Goiás.

A elicitação de requisitos a partir de reuniões é uma atividade cara, manual e suscetível à perda de informação, na qual o profissional precisa, ao mesmo tempo, participar da discussão e registrá-la em artefatos formais. Este trabalho propõe um sistema multiagente, baseado em inteligência artificial generativa, que automatiza esse processo de ponta a ponta, transformando o áudio de uma reunião em documentação de requisitos estruturada. A arquitetura é composta por quatro agentes especializados, responsáveis, respectivamente, pela transcrição automática do áudio, pela identificação de personas e geração de histórias de usuário segundo o critério INVEST, pela geração do documento de requisitos em dois formatos selecionáveis (o padrão IEEE 830 e um formato específico de cliente) e pela síntese de um diagrama de domínio. Os agentes comunicam-se por meio de um *pipeline* sequencial baseado em arquivos, o que proporciona baixo acoplamento, isolamento de falhas e flexibilidade de ponto de entrada. O sistema foi validado em um caso real, por meio de uma parceria com um órgão público, tendo como referência um documento de requisitos oficial produzido manualmente. A avaliação, de caráter quantitativo, abrange a acurácia da transcrição, a qualidade da extração de informação, a cobertura do documento gerado frente ao documento oficial e a propagação de erro e o custo ao longo do *pipeline*.

Palavras-chave

Engenharia de Requisitos; Sistemas Multiagente; Inteligência Artificial Generativa; Histórias de Usuário; Reconhecimento Automático de Fala

Abstract

Ferreira Silva, Gabriel. **Generative Artificial Intelligence and Software Engineering: Challenges and Opportunities**. Goiânia, 2026. 54p. Relatório de Graduação. Instituto de Informática, Universidade Federal de Goiás.

Requirements elicitation from meetings is an expensive, manual, and error-prone activity, in which practitioners must simultaneously participate in the discussion and record it as formal artifacts. This work proposes a multi-agent system, based on generative artificial intelligence, that automates this process end to end, transforming meeting audio into structured requirements documentation. The architecture comprises four specialized agents responsible, respectively, for the automatic transcription of the audio, the identification of personas and the generation of user stories following the INVEST criteria, the generation of the requirements document in two selectable formats (the IEEE 830 standard and a client-specific format), and the synthesis of a domain diagram. The agents communicate through a file-based sequential pipeline, which provides loose coupling, fault isolation, and entry-point flexibility. The system was validated in a real-world setting, through a partnership with a public-sector agency, using a manually produced official requirements document as ground truth. The quantitative evaluation covers transcription accuracy, information extraction quality, the coverage of the generated document against the official one, and error propagation and cost across the pipeline.

Keywords

Requirements Engineering; Multi-Agent Systems; Generative Artificial Intelligence; User Stories; Automatic Speech Recognition

Sumário

Lista de Figuras	11
1 Introdução	12
1.1 Objetivos	13
1.2 Questões de Pesquisa	14
1.3 Contribuições	14
1.4 Organização do Trabalho	15
2 Fundamentação Teórica	16
2.1 Engenharia de Requisitos e Elicitação	16
2.2 Histórias de Usuário e o Framework INVEST	17
2.3 Especificação de Requisitos de Software (IEEE 830)	17
2.4 Inteligência Artificial Generativa e Modelos de Linguagem	18
2.5 Sistemas Multiagente	19
2.6 Reconhecimento Automático de Fala	19
3 Trabalhos Relacionados	21
3.1 Modelos de Linguagem de Grande Escala na Engenharia de Requisitos	21
3.2 Reconhecimento de Fala para Transcrição de Reuniões	22
3.3 Qualidade de Histórias de Usuário e o Framework INVEST	22
3.4 Sistemas Multiagente para Engenharia de Software	23
3.5 Lacunas Identificadas	23
4 Arquitetura e Sistema Proposto	25
4.1 Visão Geral e Arquitetura do Sistema	25
4.2 Projeto dos Agentes	26
4.2.1 Agente 1: Transcrição da Reunião	26
4.2.2 Agente 2: Identificação de Personas e Extração de Histórias de Usuário	26
4.2.3 Agente 3: Geração do Documento de Requisitos	27
4.2.4 Agente 4: Síntese do Diagrama de Domínio	27
4.3 Estratégia de Seleção de Modelos	27
4.4 Estudo de Caso Real	28
5 Avaliação	29
5.1 Desenho da Avaliação	29
5.1.1 Conjuntos de Dados	29
5.1.2 Métricas e Protocolo por Questão de Pesquisa	29
5.2 Resultados e Discussão	30
5.2.1 QP1: Acurácia da Transcrição	30

5.2.2	QP2: Identificação de Personas e Histórias de Usuário	31
5.2.3	QP3: Cobertura frente ao Documento Oficial	34
5.2.4	QP4: Propagação de Erro e Custo por Agente	36
5.3	Ameaças à Validade	38
6	Conclusão	40
6.1	Contribuições	40
6.2	Limitações	41
6.3	Trabalhos Futuros	42
6.4	Disponibilidade de Artefatos	42
	Referências Bibliográficas	43
A	Prompts do Agente de Identificação	46
A.1	Mensagem de Sistema	46
A.2	Prompt de Identificação de Personas	46
A.3	Prompt de Geração de Histórias de Usuário	48
B	Prompts do Agente de Identificação	50
B.1	Mensagem de Sistema	50
B.2	Prompt de Identificação de Personas	50
B.3	Prompt de Geração de Histórias de Usuário	52
C	Contrato de Dados entre Agentes	54

Lista de Figuras

- 4.1 *Pipeline* de quatro agentes. Os agentes se comunicam por meio de um sistema de arquivos compartilhado, monitorado pela biblioteca *watchdog*. O Agente 4 é ignorado nas execuções no formato empresa.

26

Introdução

A Engenharia de Requisitos (ER) é uma fase fundamental do desenvolvimento de software, responsável por estabelecer a ponte entre as necessidades das partes interessadas e a implementação técnica do sistema. Na maioria das organizações, os requisitos são levantados em reuniões conduzidas por profissionais especializados (analistas de requisitos, *product owners* ou *scrum masters*), cujo papel é capturar com precisão discussões complexas e em constante evolução e transformá-las em artefatos formais [2]. Esse fluxo de trabalho manual introduz riscos amplamente conhecidos: a perda de informação causada pelo intervalo de tempo entre a reunião e a etapa de documentação, a sobrecarga cognitiva do analista, que precisa simultaneamente participar da discussão e registrá-la, e a inconsistência de granularidade e de formato entre os artefatos resultantes. Como os documentos de requisitos servem de base para todas as decisões subsequentes do desenvolvimento, especificações incompletas ou de baixa qualidade traduzem-se diretamente em implementações desalinhadas, retrabalho e estouro de custos.

Ferramentas comerciais de inteligência de reuniões (como Fellow.ai [7], Jamie [13] e Otter.ai) automatizam a transcrição e a extração de itens de ação, mas produzem anotações de propósito geral, e não artefatos formais de requisitos de software. A tradução da discussão de uma reunião para uma Especificação de Requisitos de Software (do inglês *Software Requirements Specification*, SRS) estruturada permanece uma etapa manual e intensiva em conhecimento especializado. Na literatura acadêmica, os estudos que aplicam Modelos de Linguagem de Grande Escala (do inglês *Large Language Models*, LLMs) à ER tratam predominantemente de subtarefas isoladas: a geração de histórias de usuário a partir de requisitos textuais já existentes [6], a extração de modelos conceituais [24] ou a classificação de requisitos. Poucos trabalhos exploram *pipelines* completos, de ponta a ponta, que partam do áudio bruto de uma reunião e cheguem a uma documentação formalmente estruturada e específica para as partes interessadas [18].

Este trabalho aborda essa lacuna por meio de uma *arquitetura multiagente* na qual quatro agentes especializados colaboram para transformar o áudio de uma reunião em documentação de requisitos. Decompor o *pipeline* em agentes, em vez de utilizar um único *prompt* monolítico de LLM, oferece vantagens concretas de engenharia: (i) se-

paração de responsabilidades, permitindo que cada agente seja desenvolvido, testado e evoluído de forma independente; (ii) *flexibilidade de ponto de entrada*, permitindo que o usuário ingresse no *pipeline* em qualquer estágio (por exemplo, a partir de uma gravação ou de uma transcrição já existente); (iii) *seleção de modelo por agente*, ajustando a capacidade e o custo do LLM à complexidade de cada tarefa; e (iv) *propagação de erro observável*, tornando possível rastrear e medir como os erros de transcrição se propagam para os artefatos a jusante, uma preocupação de engenharia de primeira ordem em sistemas agênticos.

Experimentos iniciais com os Agentes 1 e 2, conduzidos em uma etapa anterior desta pesquisa, demonstraram a viabilidade da identificação de personas e da geração de histórias de usuário sobre reuniões do *corpus* AMI. Esta monografia estende aquela implementação à sua *forma completa*, acrescentando os Agentes 3 e 4 para a geração do documento de requisitos e a síntese do diagrama de domínio, introduzindo dois formatos de saída selecionáveis (a SRS no padrão IEEE 830 e um formato específico de cliente) e conduzindo uma validação no mundo real sobre um projeto de software real do Laboratório de Inovação Goiás (LIGO). Substitui-se, ainda, a avaliação qualitativa preliminar por um conjunto de métricas quantitativas que cobre a acurácia de transcrição, métricas de extração de informação, a cobertura do documento gerado frente a um documento oficial e a análise da propagação de erro ao longo do *pipeline*.

1.1 Objetivos

O objetivo geral deste trabalho é desenvolver e avaliar um sistema multiagente, baseado em inteligência artificial generativa, capaz de automatizar a elicitação de requisitos de software a partir do áudio de reuniões, produzindo documentos de requisitos estruturados.

Para atingir esse objetivo geral, foram definidos os seguintes objetivos específicos:

1. Implementar um *pipeline* de transcrição automática de áudio de reuniões em português, robusto a arquivos longos e a diferentes formatos de mídia.
2. Identificar automaticamente as personas (atores do sistema) presentes na reunião e gerar histórias de usuário segundo o critério INVEST.
3. Gerar documentos de requisitos estruturados em dois formatos distintos (o padrão IEEE 830 e um formato específico de cliente), a partir das informações extraídas.
4. Validar o sistema em um caso real, por meio de uma parceria com o Laboratório de Inovação Goiás (LIGO), comparando o documento gerado com um documento oficial produzido manualmente (*gold standard*).

5. Avaliar quantitativamente o desempenho de cada agente e analisar empiricamente a propagação de erros entre os estágios do *pipeline*.

1.2 Questões de Pesquisa

A partir dos objetivos delineados, este trabalho busca responder às seguintes questões de pesquisa (QP):

- QP1.** Qual é a acurácia da transcrição automática (WER/CER) do Agente 1 para áudio de reuniões em português?
- QP2.** Com que precisão o Agente 2 identifica as personas das partes interessadas e gera histórias de usuário (Precisão/Revocação/F1 e conformidade com o INVEST)?
- QP3.** Quão completamente o documento gerado cobre uma especificação de requisitos oficial produzida manualmente (cobertura/Revocação)?
- QP4.** Como os erros se propagam ao longo dos estágios do *pipeline* e como a seleção de modelo por agente afeta o compromisso entre custo e qualidade?

1.3 Contribuições

As principais contribuições deste trabalho são:

1. Um sistema multiagente completo e de código aberto, composto por quatro agentes, para a elicitación de requisitos de ponta a ponta a partir do áudio de reuniões, com suporte a dois formatos de documento (a SRS no padrão IEEE 830 e um formato específico de cliente).
2. Uma validação no mundo real sobre um projeto de software de um órgão público, em parceria com o Laboratório de Inovação Goiás (LIGO), utilizando como referência (*ground truth*) um documento de requisitos oficial produzido manualmente.
3. Uma avaliação quantitativa que abrange WER/CER (Agente 1), Precisão/Revocação/F1 com concordância entre avaliadores via *kappa* de Cohen (Agente 2) e a cobertura do documento gerado frente a um documento oficial produzido manualmente (Agente 3).
4. Uma análise empírica estruturada da propagação de erro entre agentes em um *pipeline* agêntico, acompanhada de medições de latência e de custo de API por estágio.

1.4 Organização do Trabalho

O restante deste trabalho está organizado da seguinte forma. O Capítulo 2 apresenta a fundamentação teórica, revisando os conceitos essenciais ao entendimento do trabalho: engenharia de requisitos e elicitação, histórias de usuário e o *framework* INVEST, a especificação IEEE 830, modelos de linguagem de grande escala, sistemas multiagente e reconhecimento automático de fala. O Capítulo 3 discute os trabalhos relacionados, posicionando esta pesquisa frente ao estado da arte e destacando as lacunas que a motivam. O Capítulo 4 descreve a arquitetura do sistema proposto, o projeto de cada um dos quatro agentes, a estratégia de seleção de modelos e o estudo de caso real. O Capítulo 5 apresenta o desenho da avaliação, os resultados obtidos e as ameaças à validade. Por fim, o Capítulo 6 apresenta as conclusões e os trabalhos futuros.

Fundamentação Teórica

Este capítulo apresenta os conceitos fundamentais para a compreensão deste trabalho. Inicialmente, discute-se a engenharia de requisitos e a atividade de elicitação (Seção 2.1). Em seguida, abordam-se as histórias de usuário e o *framework* INVEST (Seção 2.2) e a especificação de requisitos no padrão IEEE 830 (Seção 2.3). Por fim, apresentam-se os fundamentos de inteligência artificial generativa e modelos de linguagem (Seção 2.4), de sistemas multiagente (Seção 2.5) e de reconhecimento automático de fala (Seção 2.6).

2.1 Engenharia de Requisitos e Elicitação

A Engenharia de Requisitos (ER) é o ramo da engenharia de software dedicado a descobrir, documentar e manter o conjunto de requisitos de um sistema [21]. Um requisito descreve uma capacidade ou condição que o sistema deve atender para satisfazer as necessidades das partes interessadas (*stakeholders*). É prática consolidada distinguir os *requisitos funcionais*, que especificam o que o sistema deve fazer (serviços, comportamentos e transformações de dados), dos *requisitos não-funcionais*, que impõem restrições sobre como o sistema deve operar (desempenho, segurança, usabilidade, entre outros) [27].

O processo de ER costuma ser organizado em atividades de *elicitação*, *análise e negociação*, *especificação*, *validação* e *gerência* de requisitos [21]. A elicitação é a primeira e uma das mais críticas dessas atividades: é nela que o analista interage com as partes interessadas (por meio de entrevistas, *workshops* e, sobretudo, reuniões) para descobrir e compreender suas necessidades. A literatura é consistente ao apontar que erros introduzidos nesta fase são os mais caros de corrigir, pois se propagam para todas as etapas subsequentes do desenvolvimento; quanto mais tarde um defeito de requisito é detectado, maior o custo de sua correção [27].

Na prática das organizações, a elicitação é conduzida por profissionais especializados (analistas de requisitos, *product owners* ou *scrum masters*), cuja tarefa é capturar discussões complexas e transformá-las em artefatos formais. Esse fluxo manual é trabalhoso e suscetível a falhas: há perda de informação entre a reunião e a documentação,

sobrecarga cognitiva sobre o profissional que participa e registra simultaneamente, e inconsistência de granularidade entre os artefatos resultantes. A busca por métricas e por ferramentas que apoiem e automatizem essas atividades é, portanto, um tema de interesse contínuo na área [2].

2.2 Histórias de Usuário e o Framework INVEST

No desenvolvimento ágil de software, as *histórias de usuário* (*user stories*) são o artefato predominante para a representação de requisitos [5]. Uma história de usuário é uma descrição curta de uma funcionalidade, redigida da perspectiva de quem deseja utilizá-la, tipicamente no formato “*Como [papel], quero [objetivo] para [benefício]*” [16]. Mais do que um documento, a história de usuário é um convite à conversa: ela captura a intenção de forma concisa e adia o detalhamento para o momento da implementação.

Para avaliar se uma história de usuário está bem formulada, Wake [30] propôs o conjunto de heurísticas INVEST, acrônimo de seis critérios: *Independent* (independente de outras histórias), *Negotiable* (negociável, não um contrato fechado), *Valuable* (de valor para o usuário), *Estimable* (estimável em esforço), *Small* (pequena o suficiente para caber em uma iteração) e *Testable* (testável por meio de critérios de aceitação). O critério *Small* é, na prática, um dos mais difíceis de atender, pois histórias amplas demais comprometem a estimativa e o planejamento.

A avaliação sistemática da qualidade de histórias de usuário tem sido objeto de pesquisa. Lucassen et al. [16] formalizaram o *framework Quality User Story* (QUS), com 13 critérios de sintaxe, semântica e pragmática, e a ferramenta AQUASA para a detecção automática de defeitos. Xu et al. [32], por sua vez, combinaram o INVEST com um modelo de sete critérios para automatizar essa avaliação. Neste trabalho, o critério INVEST é adotado tanto como guia para a geração das histórias quanto como base para a sua avaliação.

2.3 Especificação de Requisitos de Software (IEEE 830)

Uma Especificação de Requisitos de Software (do inglês *Software Requirements Specification*, SRS) é o documento que registra, de forma estruturada, os requisitos de um sistema, servindo de acordo entre clientes e desenvolvedores e de base para o projeto, a implementação e os testes. O padrão IEEE 830 [12] estabelece uma prática recomendada para a elaboração desse documento, descrevendo as características desejáveis de uma boa SRS (correta, não ambígua, completa, consistente, verificável, modificável e rastreável) e propondo uma estrutura de referência.

Segundo o padrão, uma SRS organiza-se, em linhas gerais, em três partes: uma *introdução* (propósito, escopo, definições e visão geral), uma *descrição geral* (perspectiva e funções do produto, características dos usuários, restrições e dependências) e os *requisitos específicos*, nos quais se detalham os requisitos funcionais e não-funcionais, frequentemente apoiados por casos de uso. Os *casos de uso* descrevem a interação entre atores e o sistema para a realização de um objetivo, complementando a especificação textual dos requisitos. Neste trabalho, o padrão IEEE 830 é um dos dois formatos de saída suportados pelo sistema proposto (Capítulo 4).

2.4 Inteligência Artificial Generativa e Modelos de Linguagem

A inteligência artificial generativa designa a classe de técnicas capazes de produzir conteúdo novo (texto, imagem, áudio ou código) a partir de padrões aprendidos de grandes volumes de dados. No domínio do processamento de linguagem natural, esse avanço é impulsionado pelos *Modelos de Linguagem de Grande Escala* (do inglês *Large Language Models*, LLMs), redes neurais com bilhões de parâmetros treinadas para prever a continuação de uma sequência de texto e que, como consequência, adquirem capacidade de compreensão e geração de linguagem em domínios variados [11]. Esses modelos têm sido aplicados a uma ampla gama de tarefas de engenharia de software, da geração de código à melhoria automática da documentação de sistemas [28].

A maior parte dos LLMs modernos baseia-se na arquitetura *Transformer*, introduzida por Vaswani et al. [29], cujo mecanismo de *autoatenção* permite ponderar a relevância de cada elemento da sequência em relação aos demais, capturando dependências de longo alcance de forma paralelizável. A interação com esses modelos se dá pela *engenharia de prompts*, isto é, a formulação cuidadosa das instruções de entrada para orientar a saída do modelo, recurso central neste trabalho, em que cada agente é especializado por meio de *prompts* próprios.

Um padrão de uso particularmente relevante para a avaliação é o de *LLM como juiz* (*LLM-as-judge*), no qual um modelo de linguagem é empregado para avaliar artefatos segundo critérios definidos, como julgar a conformidade de uma história de usuário com o INVEST. Por ser uma avaliação automática, esse padrão é escalável, mas requer validação contra avaliadores humanos para que se possa confiar em seus julgamentos, conforme adotado neste trabalho.

2.5 Sistemas Multiagente

Um *agente* é uma entidade computacional situada em um ambiente, capaz de perceber esse ambiente e de agir sobre ele de forma autônoma para atingir seus objetivos [31]. Um *sistema multiagente* (do inglês *Multi-Agent System*, MAS) é composto por múltiplos agentes que interagem entre si, por cooperação ou coordenação, para resolver problemas que estariam além da capacidade (ou da conveniência) de um único agente.

Com a popularização dos LLMs, arquiteturas multiagente passaram a ser aplicadas a tarefas de engenharia de software, atribuindo a agentes especializados papéis distintos que colaboram para produzir um resultado. Um levantamento recente de Cai et al. [3], que analisou 94 trabalhos sobre sistemas multiagente baseados em LLMs para engenharia de software, identificou a *Cooperação Baseada em Papéis (Role-Based Cooperation)* como o padrão de projeto mais adotado. A decomposição de um problema em agentes especializados, em contraste com um único *prompt* monolítico, traz vantagens de engenharia como a separação de responsabilidades, o isolamento de falhas, a possibilidade de testar e evoluir cada agente de forma independente e a seleção do modelo mais adequado (em capacidade e custo) para cada tarefa. Essas propriedades fundamentam a arquitetura adotada neste trabalho.

2.6 Reconhecimento Automático de Fala

O reconhecimento automático de fala (do inglês *Automatic Speech Recognition*, ASR) é a tarefa de converter um sinal de áudio com fala em texto. É a tecnologia habilitadora deste trabalho, pois transforma o áudio bruto de uma reunião na transcrição que alimenta os estágios seguintes. Radford et al. [23] introduziram o Whisper, um modelo treinado por supervisão fraca em larga escala que alcança transcrição multilíngue robusta sob condições acústicas adversas (ruído de fundo, sotaques e falantes não nativos), características comuns em reuniões reais.

O ASR em reuniões de elicitação, contudo, enfrenta desafios específicos: vocabulário técnico, fala sobreposta e siglas próprias de cada projeto, que modelos de propósito geral podem transcrever incorretamente. A qualidade de uma transcrição é tipicamente medida pela *Taxa de Erro de Palavra* (do inglês *Word Error Rate*, WER) e pela *Taxa de Erro de Caractere* (*Character Error Rate*, CER). A WER é definida como

$$\text{WER} = \frac{S + D + I}{N},$$

em que S , D e I são, respectivamente, o número de substituições, exclusões e inserções

necessárias para transformar a transcrição produzida na transcrição de referência, e N é o número total de palavras na referência. A CER é calculada de forma análoga, no nível de caracteres. Essas métricas fundamentam a avaliação do primeiro agente do sistema (Capítulo 5).

Trabalhos Relacionados

Este capítulo revisa os trabalhos relacionados em quatro frentes: a aplicação de modelos de linguagem de grande escala à engenharia de requisitos, o reconhecimento de fala para a transcrição de reuniões, a avaliação da qualidade de histórias de usuário e os sistemas multiagente voltados à engenharia de software. Ao final, sintetizam-se as lacunas identificadas na literatura que motivam esta pesquisa.

3.1 Modelos de Linguagem de Grande Escala na Engenharia de Requisitos

A aplicação de modelos de linguagem de grande escala à engenharia de requisitos tem atraído interesse crescente [11]. Motger et al. [18] conduziram um estudo de mapeamento sistemático que identificou 62 conjuntos de dados publicamente disponíveis para pesquisa em ER baseada em LLMs, constatando que apenas 2 deles tratam de tarefas de elicitación e que 58 dos 62 são exclusivamente em inglês, uma escassez que motiva diretamente a validação em língua portuguesa apresentada neste trabalho.

No que diz respeito à geração de artefatos, Delgado et al. [6] compararam heurísticas baseadas em *N-gram* com o GPT-3 para a geração automática de histórias de usuário a partir de especificações textuais já existentes, reportando ROUGE-N = 0,46 e BERTScore = 0,69 para a abordagem baseada em LLM. Ren et al. [24] demonstraram, ainda, que a técnica de *Chain-of-Thought* permite que LLMs extraiam diagramas de classes conceituais a partir de histórias de usuário. Naimi et al. [19] utilizaram LLMs para gerar a descrição textual de casos de uso a partir de diagramas UML codificados como XML, reduzindo o tempo de documentação e melhorando a consistência. Em conjunto, esses trabalhos demonstram a viabilidade dos LLMs para a geração de artefatos de ER, mas compartilham um pré-requisito comum: a entrada textual estruturada. Nenhum deles trata da extração de requisitos a partir do áudio não estruturado de reuniões.

3.2 Reconhecimento de Fala para Transcrição de Reuniões

O reconhecimento automático de fala (do inglês *Automatic Speech Recognition*, ASR) é a tecnologia habilitadora da elicitación de requisitos baseada em áudio. Radford et al. [23] introduziram o Whisper, um modelo treinado por supervisão fraca em larga escala que demonstra transcrição multilíngue robusta sob diversas condições acústicas, incluindo falantes não nativos e ruído de fundo, características comuns em reuniões reais de partes interessadas. Kasakowskij e Haake [15] desenvolveram o T³ Talk2Text, um sistema de ASR em tempo quase real para reuniões virtuais em grupo, que captura transcrições de todo o grupo para a análise posterior de padrões de comunicação e colaboração.

Apesar desses avanços, o ASR em reuniões de elicitación de requisitos enfrenta desafios específicos do domínio: vocabulário técnico, fala sobreposta e siglas específicas de projeto que modelos de propósito geral podem transcrever incorretamente. Tais erros não permanecem isolados: eles se propagam para os agentes a jusante, um modo de falha que se analisa empiricamente na QP4.

3.3 Qualidade de Histórias de Usuário e o Framework INVEST

As histórias de usuário são o artefato de requisitos predominante no desenvolvimento ágil, seguindo o formato “*Como [papel], quero [objetivo] para [benefício]*” [16]. Wake [30] propôs as heurísticas INVEST (*Independent, Negotiable, Valuable, Estimable, Small e Testable*) como seis critérios para avaliar se uma história de usuário é acionável e bem delimitada.

Lucassen et al. [16] formalizaram o *framework* de *Quality User Story* (QUS), composto por 13 critérios que cobrem sintaxe, semântica e pragmática, e introduziram a AQUSA, uma ferramenta baseada em processamento de linguagem natural que detecta defeitos de qualidade. Avaliada sobre 1.023 histórias de 18 organizações de software, a AQUSA demonstra a viabilidade da avaliação automatizada de qualidade, mas requer histórias escritas manualmente como entrada. Xu et al. [32] combinaram o INVEST com um modelo de sete critérios para automatizar a avaliação de qualidade em *pipelines* ágeis. Este trabalho estende essa linha ao gerar histórias de usuário a partir do áudio de reuniões e ao avaliá-las frente ao INVEST por meio de um protocolo de LLM como juiz validado contra avaliadores humanos.

3.4 Sistemas Multiagente para Engenharia de Software

Arquiteturas multiagente emergiram como um paradigma produtivo para a automação de tarefas complexas de engenharia de software. O ChatDev [22] introduziu um *framework* baseado em diálogo no qual agentes LLM especializados colaboram por meio de conversas estruturadas ao longo das fases de projeto, codificação e teste, demonstrando que a comunicação linguística pode unificar agentes autônomos. O MetaGPT [10] codificou Procedimentos Operacionais Padronizados (SOPs) em sequências de *prompts*, atribuindo papéis de engenharia de software (gerente de produto, arquiteto, engenheiro e responsável por qualidade) a agentes distintos, alcançando resultados do estado da arte em *benchmarks* de geração de código. Luo et al. [17] aplicaram um *pipeline* agêntico à documentação, com três agentes especializados produzindo e mantendo a documentação de código em nível de repositório, superando as linhas de base produzidas por humanos. Há, ainda, esforços rumo a plataformas unificadas de desenvolvimento multiagente [25] e à automação de fluxos ágeis completos, da criação de histórias de usuário à geração e revisão de código [26]. Um levantamento recente de Cai et al. [3] analisou 94 artigos sobre sistemas multiagente baseados em LLMs para tarefas de engenharia de software, constatando que a *Cooperação Baseada em Papéis (Role-Based Cooperation)* é o padrão de projeto mais comumente adotado, o mesmo padrão que fundamenta o *pipeline* proposto neste trabalho.

O trabalho mais próximo do nosso é o MARE [14], um *framework* de colaboração multiagente para todo o processo de ER. O MARE decompõe a ER em quatro tarefas (elicitação, modelagem, verificação e especificação) distribuídas entre cinco agentes especializados, superando as linhas de base do estado da arte em até 15,4% em F1. Contudo, o MARE opera a partir de uma *ideia inicial (rough idea)* textual fornecida por um humano e simula a elicitação por meio de diálogo entre agentes; ele não processa o áudio de reuniões e, portanto, não pode ser implantado em um cenário pós-reunião totalmente automatizado. O sistema proposto neste trabalho aborda precisamente essa lacuna.

3.5 Lacunas Identificadas

A literatura revisada revela quatro lacunas que motivam este trabalho:

1. **Ausência de *pipelines* de áudio para SRS.** As ferramentas comerciais existentes [7, 13] produzem anotações de reunião de propósito geral, e não artefatos formais de ER. Os sistemas de ER baseados em LLMs, como o MARE [14], processam texto estruturado, e não o áudio conversacional bruto.
2. **Foco em subtarefas isoladas.** O trabalho anterior com LLMs em ER trata de tarefas individuais (geração de histórias de usuário [6], descrição de casos de uso [19],

modelagem conceitual [24]) sem integrar a transcrição, a identificação de personas e a geração de documentos em um único *pipeline*.

3. **Falta de validação no mundo real.** A maioria dos estudos utiliza conjuntos de dados sintéticos ou *corpora* controlados; poucos avaliam o resultado frente a um documento de requisitos oficial produzido manualmente em um projeto de software real.
4. **Propagação de erro não analisada.** Em *pipelines* agênticos, os erros de transcrição se propagam em cascata para os artefatos a jusante. Esse modo de falha é reconhecido, mas não é medido empiricamente em nenhum dos trabalhos relacionados.

Este trabalho aborda as quatro lacunas por meio de um *pipeline* de quatro agentes que transforma o áudio de reuniões em documentação de requisitos estruturada, validado sobre um projeto de software governamental real e com a medição quantitativa da propagação de erro entre agentes.

Arquitetura e Sistema Proposto

Este capítulo descreve a arquitetura do sistema proposto, o projeto de cada um dos quatro agentes especializados, a estratégia de seleção de modelos por agente e o estudo de caso real utilizado para a validação.

4.1 Visão Geral e Arquitetura do Sistema

O sistema proposto é composto por quatro agentes especializados conectados por meio de um *pipeline* sequencial baseado em arquivos, ilustrado na Figura 4.1. Cada agente monitora o diretório de saída de seu predecessor utilizando a biblioteca *watchdog*; ao detectar um novo arquivo, dispara o processamento automaticamente, sem intervenção manual. Esse projeto proporciona baixo acoplamento entre os estágios, isolamento de falhas e artefatos intermediários auditáveis. O usuário pode ingressar no *pipeline* em qualquer estágio (fornecendo o áudio bruto, uma transcrição já existente ou um arquivo JSON de análise anterior), o que permite o reaproveitamento parcial de resultados entre execuções.

O *pipeline* suporta dois formatos de saída selecionáveis, escolhidos uma única vez por execução: **IEEE 830**, a estrutura padrão de Especificação de Requisitos de Software utilizada ao longo desta pesquisa; e **formato empresa**, um *layout* de documento específico de cliente, solicitado pelo parceiro (Seção 4.4). Nas execuções no formato empresa, o Agente 4 é ignorado, pois esse formato de cliente não inclui diagramas de domínio. O formato afeta apenas o *template* do documento e alguns *prompts* dos agentes; todos os estágios anteriores (transcrição, identificação de personas e geração de histórias de usuário) são agnósticos ao formato.

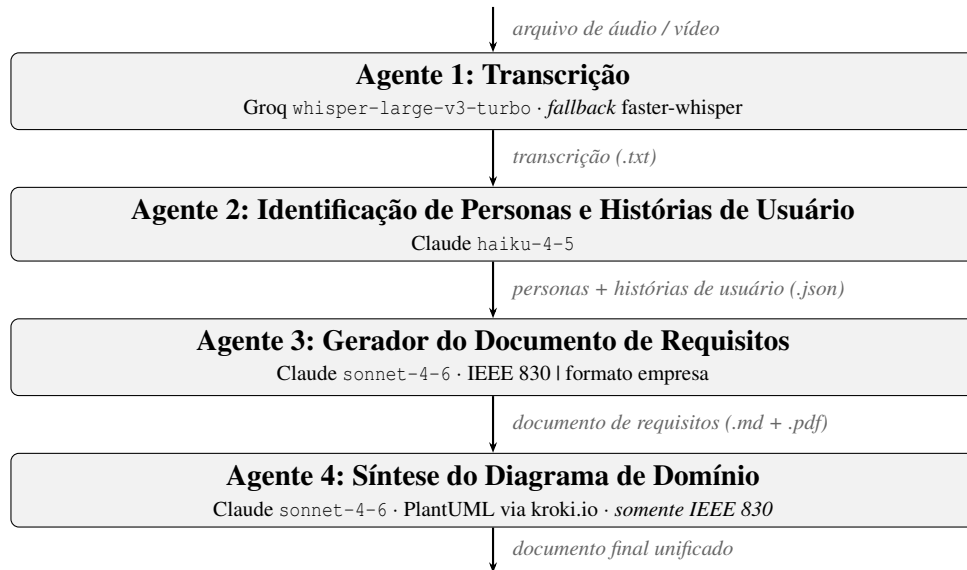


Figura 4.1: Pipeline de quatro agentes. Os agentes se comunicam por meio de um sistema de arquivos compartilhado, monitorado pela biblioteca watchdog. O Agente 4 é ignorado nas execuções no formato empresa.

4.2 Projeto dos Agentes

4.2.1 Agente 1: Transcrição da Reunião

O Agente 1 aceita qualquer arquivo de áudio ou vídeo e produz uma transcrição em texto puro. O processamento começa com a normalização via **ffmpeg**: a entrada é convertida em um arquivo FLAC mono de 16 kHz, reduzindo o tamanho e preservando o sinal relevante para a transcrição. Se o arquivo normalizado exceder o limite de tamanho da API da Groq, ele é dividido em segmentos nas fronteiras de silêncio, cada um transcrito de forma independente e, ao final, mesclado em uma única saída.

A transcrição é realizada pela **Groq** executando o modelo `whisper-large-v3-turbo` [23, 9], escolhido por combinar alta acurácia multilíngue e baixa latência por meio de inferência acelerada por *hardware*. Caso a API da Groq esteja indisponível, o agente recorre automaticamente ao **faster-whisper**, executando o modelo `small` localmente [20], trocando acurácia por disponibilidade *offline*.

4.2.2 Agente 2: Identificação de Personas e Extração de Histórias de Usuário

O Agente 2 lê a transcrição e executa duas chamadas sequenciais a um LLM, utilizando o **Claude haiku-4-5** [1]. A primeira chamada identifica as personas distintas das partes interessadas presentes na conversa, extraindo, para cada uma: nome ou identificador, papel, características definidoras e trechos representativos de fala. A segunda

chamada gera histórias de usuário específicas de cada persona, seguindo o critério INVEST [30] no formato “*Como [persona], quero [objetivo] para [benefício]*”. Decompor a identificação e a geração em duas chamadas reduz a interferência de contexto e melhora a estruturação da saída. Os resultados são salvos em um documento JSON tipado, que serve de contrato entre os Agentes 2 e 3.

4.2.3 Agente 3: Geração do Documento de Requisitos

O Agente 3 lê o JSON produzido pelo Agente 2 e gera um documento de requisitos utilizando o **Claude sonnet-4-6** com *templates Jinja2*. No modo IEEE 830, a saída segue a estrutura padrão de SRS (introdução, descrição geral, partes interessadas, requisitos funcionais e tabela de histórias de usuário). No modo empresa, segue o *layout* do cliente (funcionalidades, requisitos funcionais detalhados com comentários, regras de negócio, casos de uso e requisitos não-funcionais). O documento é renderizado em *Markdown* e, em seguida, em PDF por meio do **WeasyPrint**, utilizando *CSS Paged Media* para gerar um sumário com numeração automática de páginas.

4.2.4 Agente 4: Síntese do Diagrama de Domínio

O Agente 4 só é ativado nas execuções no formato IEEE 830. Ele lê tanto o documento do Agente 3 quanto o JSON original do Agente 2 e utiliza o **Claude sonnet-4-6** para extrair as entidades de domínio e seus relacionamentos. O diagrama de classes em PlantUML resultante é renderizado em PNG por meio da API pública **kroki.io** e incorporado a um documento de requisitos final unificado, que consolida todas as seções anteriores.

4.3 Estratégia de Seleção de Modelos

Cabe destacar, de antemão, que a escolha da família de modelos empregada neste trabalho, o Claude, da Anthropic, para os agentes baseados em LLM, e o Whisper, disponibilizado pela Groq, para a transcrição, deveu-se essencialmente a questões de disponibilidade, e não a uma comparação sistemática entre provedores. Trata-se de modelos que o autor já utiliza diariamente e para os quais já dispunha de créditos e de experiência prática de uso, o que reduziu o custo e o tempo de configuração do *pipeline*. A arquitetura proposta é, contudo, agnóstica ao provedor: cada agente encapsula a chamada ao modelo em um cliente próprio, de modo que a substituição por modelos equivalentes de outros fornecedores exigiria apenas mudanças pontuais de configuração.

Feita essa ressalva, a seleção do *tier* de modelo dentro da família adotada, por agente, reflete um compromisso explícito entre custo e qualidade. O Agente 2 realiza

extração estruturada com um esquema de saída bem definido (JSON tipado); a tarefa é repetitiva e executada em todas as reuniões, o que torna o custo por chamada uma preocupação primária, daí o uso do modelo mais leve `haiku-4-5`. Os Agentes 3 e 4 realizam síntese aberta e geração de documentos, que exigem raciocínio mais forte e fluência de linguagem; para ambos, utiliza-se o `sonnet-4-6`. O Agente 1 emprega um modelo de ASR dedicado (Whisper) em vez de um LLM de propósito geral, pois o reconhecimento de fala é uma tarefa especializada na qual o Whisper alcança acurácia e velocidade substancialmente melhores.

4.4 Estudo de Caso Real

Para validar o sistema em um cenário autêntico, estabeleceu-se uma parceria com o **Laboratório de Inovação Goiás (LIGO)** [8], instituído pela Lei estadual nº 23.042, de 29 de outubro de 2024, por meio de sua Gerência de Inovação, Ciência de Dados e Inteligência Artificial. A equipe forneceu uma reunião gravada de partes interessadas e um documento de requisitos oficial, produzido manualmente para o mesmo sistema, que serve como referência (*ground truth*) para a avaliação (QP3–QP4).

A reunião envolveu **10 participantes** e teve duração de **1 hora, 37 minutos e 47 segundos**, conduzida inteiramente em português brasileiro. O tema foi a reformulação de um módulo de gestão de saldo orçamentário em um sistema governamental de planejamento financeiro. O arquivo de vídeo bruto (609 MB) foi processado de ponta a ponta pelo *pipeline*: normalização via `ffmpeg`, segmentação por silêncio, transcrição pela Groq, identificação de personas e geração do documento no formato empresa.

O formato empresa foi adotado a pedido do parceiro, uma vez que a organização não utiliza o IEEE 830 internamente. Esse requisito motivou a arquitetura de formato duplo: os estágios centrais de extração de dados são agnósticos ao formato, e o formato do documento é um parâmetro de saída configurável. O documento gerado e o documento oficial de referência são comparados quantitativamente no Capítulo 5.

Avaliação

Este capítulo apresenta a avaliação do sistema proposto. A Seção 5.1 descreve o desenho da avaliação: os conjuntos de dados, as métricas e o protocolo associado a cada questão de pesquisa. A Seção 5.2 reporta os resultados obtidos, e a Seção 5.3 discute as ameaças à validade do estudo.

5.1 Desenho da Avaliação

A avaliação foi estruturada em torno das quatro questões de pesquisa definidas no Capítulo 1, cada uma associada a um agente ou ao *pipeline* como um todo. Esta seção descreve, para cada questão, os dados utilizados, as métricas adotadas e o protocolo de medição.

5.1.1 Conjuntos de Dados

Três fontes de dados sustentam a avaliação:

- **Corpus AMI:** conjunto de reuniões anotadas [4], utilizado para a avaliação da identificação de personas e da geração de histórias de usuário (QP2). Sete reuniões foram processadas na etapa inicial desta pesquisa.
- **Corpus de áudio em português:** utilizado para a medição da acurácia de transcrição (QP1), que exige pares de áudio e transcrição de referência em português.
- **Estudo de caso real (parceiro público):** a reunião gravada e o documento de requisitos oficial descritos na Seção 4.4, base da questão de cobertura (QP3) e do estudo de propagação de erro e custo (QP4).

5.1.2 Métricas e Protocolo por Questão de Pesquisa

QP1: Acurácia da transcrição (Agente 1). Mede-se a Taxa de Erro de Palavra (WER) e a Taxa de Erro de Caractere (CER) da transcrição produzida frente à transcrição de referência, sobre o corpus de áudio em português. Reporta-se, ainda, o *Real-Time Factor*

(RTF), razão entre o tempo de processamento e a duração do áudio, e compara-se a transcrição via Groq com a execução local do *faster-whisper*.

QP2: Identificação de personas e histórias de usuário (Agente 2). Avalia-se a identificação de personas por meio de Precisão, Revocação e F1, comparando as personas (atores do sistema) identificadas com um gabarito de referência das reuniões. A qualidade das histórias de usuário é medida pela taxa de conformidade com os seis critérios INVEST, aplicada por um protocolo de LLM como juiz, tanto sobre o *corpus* AMI quanto sobre as histórias do estudo de caso real. Como a avaliação automática é sensível ao modelo empregado, comparam-se dois juízes (um modelo leve e um modelo de raciocínio) e adota-se o segundo como referência, validando-o em ambos os domínios por anotação manual cega. A concordância entre o juiz de referência e o anotador humano é quantificada pelo *kappa* de Cohen.

QP3: Cobertura frente ao documento oficial (Agente 3). Compara-se o documento gerado com o documento oficial produzido manualmente (*gold standard*), por meio de um protocolo de casamento item a item dos requisitos. Como o documento oficial detalha apenas um item do projeto (item 2.1), a comparação restringe-se a esse escopo comparável. A métrica central é a Revocação (cobertura), ou seja, a fração dos requisitos do documento oficial efetivamente capturada pelo documento gerado, calculada separadamente para requisitos funcionais, regras de negócio e casos de uso. O casamento é conduzido por anotação manual.

QP4: Propagação de erro e custo por agente (*pipeline*). A propagação de erro é analisada como um estudo de caso qualitativo, rastreando como um erro de transcrição se manifesta nos artefatos a jusante até o documento final. Complementarmente, medem-se o tempo de processamento por estágio, o tempo total por reunião e o custo de API (estimado a partir do número de *tokens* consumidos por cada agente), oferecendo evidência sobre a viabilidade prática do sistema.

5.2 Resultados e Discussão

5.2.1 QP1: Acurácia da Transcrição

A acurácia da transcrição do Agente 1 foi medida sobre o *corpus* Mozilla Common Voice em português, comparando-se a transcrição produzida pela Groq (*whisper-large-v3-turbo*) com a execução local do *faster-whisper*, sobre uma amostra de 200 clipes. A Tabela 5.1 reporta a WER e a CER em nível de *corpus* (agregadas

sobre o total de palavras e caracteres, e não como média de razões por amostra), além do *Real-Time Factor* (RTF) médio. Para que diferenças de formatação não fossem contabilizadas como erros de transcrição, referência e hipótese foram normalizadas (minúsculas, sem pontuação e com números por extenso convertidos a dígitos) antes do cálculo das métricas.

Motor	Amostras	WER	CER	RTF médio
Groq (<i>whisper-large-v3-turbo</i>)	200	6,3%	2,3%	0,91
Local (<i>faster-whisper, small</i>)	200	13,5%	4,9%	0,56

Tabela 5.1: Acurácia da transcrição (WER/CER em nível de corpus) e Real-Time Factor (RTF) médio do Agente 1, sobre 200 clipes do corpus Common Voice em português.

O motor Groq alcançou uma WER de 6,3% e uma CER de 2,3%, patamar compatível com o estado da arte do Whisper para o português e suficiente para alimentar os estágios seguintes do *pipeline*. Setenta e seis por cento dos clipes foram transcritos sem nenhum erro; os erros remanescentes são majoritariamente fonéticos (por exemplo, “vinhedo” transcrito como “vinhedos”), e não falhas grosseiras. O motor local (*faster-whisper* no preset *small*) apresentou WER mais que duas vezes maior (13,5%), o que evidencia o compromisso entre **acurácia e disponibilidade**: a execução local serve como alternativa *offline* quando a API está indisponível, ao custo de uma queda de qualidade.

Quanto à eficiência, ambos os motores operaram com RTF abaixo de 1,0 (0,91 para a Groq e 0,56 para o local), isto é, transcreveram mais rápido que o tempo real. Vale notar que, sobre clipes curtos do Common Voice (com duração média de poucos segundos), o RTF é dominado pela latência de rede por requisição; em arquivos longos, como a reunião do estudo de caso real, a Groq atingiu um RTF de aproximadamente 0,018 (Seção 5.2.4), muito mais favorável.

Duas ressalvas metodológicas acompanham esses resultados. Primeiro, o Common Voice é composto por fala lida em clipes curtos, e não por fala espontânea e contínua de reuniões; assim, a WER aqui reportada tende a **subestimar** a dificuldade de um cenário real, no qual há sobreposição de falas, ruído e termos técnicos. Segundo, parte dos erros em qualquer corpus decorre de vocabulário específico de domínio: o caso “IPOF” transcrito como “HIPOF” na reunião real (analisado na QP4) ilustra como uma sigla de projeto, ausente do treinamento do modelo, pode ser transcrita incorretamente e propagar-se a jusante.

5.2.2 QP2: Identificação de Personas e Histórias de Usuário

Esta subseção avalia o Agente 2 em duas dimensões: a identificação das personas (atores do sistema) e a qualidade das histórias de usuário geradas, ambas sobre as sete

reuniões processadas do *corpus* AMI.

Identificação de personas. A Tabela 5.2 apresenta a Precisão, a Revocação e o F1 da identificação de personas, comparando os papéis identificados pelo agente com os papéis de referência das reuniões. O casamento entre o papel previsto e o papel de referência segue o protocolo de normalização e resolução de sinônimos descrito na Seção 5.1; os valores são micromediados sobre as sete reuniões.

Métrica	Precisão	Revocação	F1
Identificação de personas (micromédia)	1,00	0,92	0,96

Tabela 5.2: *Precisão, Revocação e F1 da identificação de personas (atores do sistema) pelo Agente 2, micromediados sobre as sete reuniões do corpus AMI.*

Os resultados indicam alta precisão (1,00) e boa revocação (0,92): o agente não introduz atores espúrios e recupera a maior parte dos atores do sistema discutidos, deixando de identificar apenas perfis pouco enfatizados na conversa (por exemplo, o ator “comprador” em uma das reuniões).

Conformidade com o INVEST. A qualidade das 122 histórias de usuário geradas foi avaliada pela taxa de conformidade com os seis critérios INVEST, por meio de um protocolo de LLM como juiz, no qual o modelo classifica cada história quanto a cada critério. Considerou-se conforme a história que satisfaz simultaneamente os seis critérios.

Um achado metodológico precede a apresentação dos resultados. A taxa de conformidade mostrou-se **altamente sensível à escolha do modelo-juiz**. Avaliando o mesmo conjunto de histórias com dois juizes (o modelo mais leve *haiku-4-5* e o modelo de raciocínio *sonnet-4-6*, este último com instruções de avaliação mais detalhadas), a conformidade global passou de apenas 5,9% para 52,5%, conforme a Tabela 5.3. A inspeção das discordâncias revelou que o juiz mais leve subavaliava sistematicamente os critérios *Estimable*, *Small* e *Testable*, classificando como “não testáveis” histórias que admitem critérios de aceitação qualitativos (por exemplo, “*como telespectador, quero usar o controle confortavelmente, seja destro ou canhoto*”, verificável por avaliação de conforto com ambos os grupos). Adota-se, portanto, o juiz *sonnet-4-6* como avaliador de referência, e a comparação é reportada como evidência do compromisso entre custo e confiabilidade na avaliação automática.

Sob o juiz de referência, as histórias geradas mostram-se sobretudo **valiosas** (98,4%) e **independentes** (100%), o que indica que o agente captura necessidades reais dos atores do sistema, bem delimitadas entre si. Os critérios de **granularidade** (*Estimable*, *Small* e *Testable*, entre 73,8% e 77,9%) concentram as não-conformidades: parte das histórias ainda é ampla demais para constituir, por si só, um item acionável,

Crítério INVEST	Juiz haiku-4-5	Juiz sonnet-4-6
Independente (<i>Independent</i>)	76,3%	100,0%
Negociável (<i>Negotiable</i>)	90,7%	82,0%
Valiosa (<i>Valuable</i>)	96,6%	98,4%
Estimável (<i>Estimable</i>)	17,8%	73,8%
Pequena (<i>Small</i>)	21,2%	77,9%
Testável (<i>Testable</i>)	16,1%	77,9%
Conforme (todos os seis)	5,9%	52,5%

Tabela 5.3: Taxa de conformidade das histórias de usuário com os critérios INVEST, sobre as 122 histórias geradas, avaliada por dois modelos-juiz. O juiz *sonnet-4-6* é adotado como referência.

confirmando que o critério *Small* é o modo de falha predominante, como antecipado na fundamentação. A conformidade global de 52,5% deve ser lida à luz de seu caráter estrito (exige os seis critérios simultaneamente).

Para verificar a confiabilidade do juiz de referência, uma amostra aleatória de 25 histórias foi anotada manualmente, de forma cega ao veredito do modelo. As taxas globais de conformidade dos dois avaliadores ficaram próximas (60% para o anotador humano e 48% para o juiz na amostra), corroborando a estimativa de conformidade do conjunto completo. Em contraste, a concordância *história a história* no rótulo de conformidade plena foi de *kappa* de Cohen igual a 0,13 (concordância leve). A análise das discordâncias ajuda a entender essa diferença. Os avaliadores concordam fortemente sobre o que entrega valor: o critério *Independent* obteve concordância perfeita e o critério *Valuable* obteve concordância de 92% (humano e juiz consideram valiosa a quase totalidade das histórias). A divergência concentra-se na **testabilidade** de requisitos de natureza estética. O anotador humano aplicou um critério mais rigoroso de verificabilidade: uma história como “*quero um controle remoto com design moderno e atraente*” descreve uma necessidade legítima (valiosa), porém sua satisfação não admite um critério de aceitação objetivo, pois “ser atraente” é subjetivo, logo não é testável e constitui um requisito fraco, no sentido do IEEE 830. (Em contraste, restrições com referência objetiva, como “manter o custo dentro do orçamento”, permanecem testáveis.) O juiz automático foi mais permissivo, aceitando os requisitos estéticos como conformes. Essa divergência é, portanto, conceitual e defensável, e não um simples ruído de anotação: parte da não-conformidade reflete um padrão de qualidade legítimo do avaliador humano. Discute-se essa subjetividade como ameaça à validade na Seção 5.3.

Conformidade no domínio real. O *corpus* AMI, embora útil, é composto por reuniões de *design* de um produto de consumo (um controle remoto), o que favorece histórias de natureza aspiracional e subjetiva. Para avaliar a conformidade em um cenário de requisitos

de software genuíno, aplicou-se o mesmo protocolo às 24 histórias geradas a partir da reunião do estudo de caso real (Seção 4.4). A Tabela 5.4 contrasta os dois domínios sob o juiz de referência. No domínio real, a conformidade global sobe para **66,7%**, com 100% de histórias *testáveis* e *valiosas* e 96% *estimáveis*.

Critério INVEST	AMI (controle remoto)	Caso real (saldo orçamentário)
Independente (<i>Independent</i>)	100,0%	87,5%
Negociável (<i>Negotiable</i>)	82,0%	87,5%
Valiosa (<i>Valuable</i>)	98,4%	100,0%
Estimável (<i>Estimable</i>)	73,8%	95,8%
Pequena (<i>Small</i>)	77,9%	87,5%
Testável (<i>Testable</i>)	77,9%	100,0%
Conforme (todos os seis)	52,5%	66,7%

Tabela 5.4: Conformidade INVEST por domínio, sob o juiz de referência (*sonnet-4-6*): o corpus AMI (122 histórias) e o estudo de caso real (24 histórias).

A diferença é coerente com a natureza dos domínios: requisitos de um sistema corporativo real (“consultar o saldo disponível por ação, grupo de despesa e fonte de recurso”) são concretos e verificáveis por construção, enquanto necessidades de *design* de produto (“um controle confortável e atraente”) são mais difusas. A validação humana cega das 24 histórias do caso real reforça esse quadro: a concordância história a história entre o anotador humano e o juiz foi de *kappa* de Cohen igual a 0,31 (concordância razoável), mais do que o dobro da obtida no AMI ($\kappa = 0,13$), e a concordância observada subiu de 56% para 75%. Em um domínio de requisitos concretos, portanto, não apenas o sistema gera histórias de maior qualidade, como avaliadores humano e automático **convergem mais** sobre o que constitui uma boa história. Vale ainda uma observação qualitativa do estudo de caso: algumas histórias geradas são semanticamente próximas entre si, mas descrevem a mesma capacidade para **atores distintos** (por exemplo, a consulta de saldo solicitada pelo Gestor de Orçamento e pelo Consultor de Saldo); não se trata de redundância, mas da granularidade correta de uma história de usuário, que é sempre escrita do ponto de vista de um único ator.

5.2.3 QP3: Cobertura frente ao Documento Oficial

Esta análise compara o documento gerado pelo sistema (no formato empresa) com o documento de requisitos oficial do parceiro (PR0102), produzido manualmente por um analista e adotado como referência (*gold standard*). A apresentação combina uma análise qualitativa da convergência entre os dois documentos com a quantificação da cobertura (Revocação) no escopo comparável, apresentada ao final desta subseção.

Escopo comparável. Um cuidado metodológico precede a comparação: o documento oficial detalha **apenas um item** do projeto (a tela de Manutenção de Saldo, item 2.1), enquanto a reunião (e, por consequência, o documento gerado) abrange **todo o escopo** discutido (itens 2.1 a 2.7). Uma comparação por contagem bruta de requisitos seria, portanto, enganosa. A avaliação de cobertura deve separar o **escopo comparável** (item 2.1, presente nos dois documentos) do **escopo adicional** (itens 2.2 a 2.7, presentes apenas no documento gerado).

Convergência no núcleo funcional. No escopo comparável, os dois documentos identificaram os mesmos pontos centrais discutidos na reunião, entre eles:

- a granularidade multidimensional do saldo (exercício, unidade, ação orçamentária, grupo de despesa e fonte de recurso);
- as operações de inclusão, edição e consulta de saldo;
- a multisseleção de ação, grupo e fonte; e
- a sinalização (*flag*) de inclusão de saldo com valor zerado ou sem dotação.

Este último ponto (um detalhe bastante específico) foi capturado por ambos os documentos, o que constitui forte evidência de que o sistema extraiu com precisão o conteúdo efetivamente discutido na reunião.

Cobertura adicional do documento gerado. Por refletir toda a reunião, o documento gerado contempla requisitos ausentes do documento oficial por pertencerem a outros itens de escopo: a validação de saldo nos pontos de consumo (envio, alteração, homologação e autorização), o tratamento de exercícios futuros e os limites e o monitoramento da LOA. Trata-se de cobertura legítima, e não de ruído.

Limitação de cobertura do documento gerado. Em sentido inverso, o documento oficial contém detalhes que o gerado não reproduz, todos relativos à **concretude da interface**: o caminho de menu, os títulos de tela, os filtros campo a campo, as colunas da grade, a paginação e a integração com sistemas externos. Esses detalhes derivam de protótipos visuais, ausentes do áudio da reunião. Não se trata, portanto, de uma falha do sistema, mas de uma limitação do insumo, mitigável no futuro com entrada multimodal (anexação de telas e protótipos).

Fidelidade terminológica. Por fim, cabe registrar um aspecto de fidelidade terminológica: a transcrição do Agente 1 grafou incorretamente a sigla “IPOF” como “hipófis” em vários trechos. Esse erro, contudo, não chegou ao documento final, tendo sido corrigido pelos agentes a jusante a partir do contexto. O caso é analisado em detalhe na propagação

de erro (QP4) e ilustra tanto o risco de propagação quanto a capacidade de mitigação do *pipeline*.

Cobertura quantitativa no escopo comparável. Sobre o escopo comparável (item 2.1), conduziu-se um casamento item a item entre os requisitos do documento oficial e os do documento gerado: cada requisito do documento oficial (adotado como *gold standard*) foi classificado como coberto ou não coberto por algum requisito do documento gerado. A métrica central é a **Revocação** (cobertura), isto é, a fração dos requisitos do documento oficial efetivamente capturada pelo sistema, reportada na Tabela 5.5 por categoria de requisito. O documento oficial contém, no item 2.1, cinco requisitos funcionais, três regras de negócio e três casos de uso.

Categoria de requisito	Cobertos	Total	Revocação
Requisitos funcionais	2	5	40,0%
Regras de negócio	3	3	100,0%
Casos de uso	3	3	100,0%
Global	8	11	72,7%

Tabela 5.5: Cobertura (Revocação) do documento gerado frente ao documento oficial (*gold standard*) no escopo comparável (item 2.1), por categoria de requisito.

A cobertura global foi de **72,7%**, com 100% nas regras de negócio e nos casos de uso. A aparente baixa cobertura dos requisitos funcionais (40%) merece exame: os três requisitos funcionais não cobertos (RF2, RF4 e RF5 do documento oficial) são todos do tipo “*excluir a tela X*”, isto é, operações de manipulação de telas específicas da interface. Capturá-los exigiria que o sistema percebesse **o que estava sendo exibido na tela durante a reunião** (um protótipo ou a tela do sistema em uso), uma entrada visual que a arquitetura atual, restrita ao áudio, não processa. Não se trata, portanto, de falha de extração, mas de uma limitação de **modalidade** do insumo. Excluindo-se esses três requisitos de pura manipulação de interface, a cobertura do conteúdo funcional efetivamente discutível por voz aproxima-se de 100%. Esse resultado é coerente com a análise qualitativa: o sistema recupera fielmente o conteúdo **falado** na reunião, e as lacunas concentram-se no que só existe **visualmente**.

5.2.4 QP4: Propagação de Erro e Custo por Agente

Em um *pipeline* sequencial de agentes, um erro cometido em um estágio inicial pode propagar-se para todos os artefatos subsequentes. Esta análise documenta um caso concreto observado no estudo de caso real e estabelece o protocolo para a medição de custo e latência.

Estudo de caso de propagação de erro. Durante o processamento da reunião do parceiro, o Agente 1 transcreveu incorretamente a sigla “IPOF” (Instrumento de Planejamento, Orçamento e Finanças), grafando-a como “hipófis” em diversos trechos da transcrição. Esse é o ponto crítico de um *pipeline* sequencial: um erro do primeiro estágio fica disponível para contaminar todos os artefatos a jusante. O comportamento observado, contudo, foi mais sutil. Em uma execução anterior, na qual o Agente 3 utilizava um modelo mais leve, o termo incorreto sobreviveu até o documento de requisitos final. Já na configuração atual, com o Agente 3 baseado em um modelo de raciocínio (sonnet-4-6), os agentes a jusante **corrigiram** a sigla pelo contexto, e o documento final registrou “IPOF” corretamente. O caso ilustra dois pontos centrais sobre sistemas agênticos: a acurácia do primeiro agente (medida pela WER/CER na QP1) condiciona a qualidade potencial de todo o *pipeline*, mas agentes intermediários suficientemente capazes podem atuar como uma camada de **mitigação**, recuperando o termo correto a partir do contexto do domínio. O achado motiva, como trabalho futuro (Capítulo 6), tanto a adoção de um dicionário de termos do domínio aplicado após a transcrição quanto a investigação sistemática da capacidade de correção dos agentes a jusante.

Custo e latência por agente. Para avaliar a viabilidade prática do sistema, mediram-se o tempo de processamento e o custo de API por estágio, sobre a reunião do estudo de caso real (1 hora, 37 minutos e 47 segundos de áudio, a partir de um vídeo bruto de 609 MB). A Tabela 5.6 consolida, por agente, o tempo de processamento (tempo de relógio) e o custo estimado. O Agente 1 (Groq Whisper) é tarifado por hora de áudio (US\$ 0,04/h); os Agentes 2 e 3 são tarifados pelo número de *tokens* de entrada e de saída, segundo os preços vigentes do provedor (Claude haiku-4-5: US\$ 1,00/MTok de entrada e US\$ 5,00/MTok de saída; sonnet-4-6: US\$ 3,00/MTok e US\$ 15,00/MTok).

Agente	Modelo	Tempo (s)	Custo (US\$)
Agente 1 (Transcrição)	whisper-large-v3-turbo	107	0,065
Agente 2 (Identificação)	claude-haiku-4-5	71	0,063
Agente 3 (Documento)	claude-sonnet-4-6	499	0,593
Total por reunião	—	677	0,721

Tabela 5.6: Tempo de processamento e custo estimado de API por agente, sobre a reunião do estudo de caso real (formato empresa, em que o Agente 4 não é executado).

O *pipeline* completo processou a reunião de quase 1 hora e 40 minutos em aproximadamente **11 minutos** (677 s) e ao custo de **US\$ 0,72**, o que evidencia a viabilidade prática do sistema. Destaca-se a eficiência da transcrição: o Agente 1 transcreveu o áudio com um *Real-Time Factor* (RTF) de cerca de **0,018**, isto é, aproximadamente 55 vezes mais rápido que a duração do áudio, ao custo de apenas US\$ 0,065. O custo e o tempo

concentram-se no Agente 3 (82% do custo total e 74% do tempo), responsável pela síntese aberta do documento de requisitos com o modelo *sonnet-4-6*. Esse perfil confirma e justifica a estratégia de seleção de modelos por agente (Capítulo 4): emprega-se o modelo mais leve (*haiku-4-5*) na extração estruturada repetitiva do Agente 2, reservando o modelo de raciocínio mais caro (*sonnet-4-6*) para a geração do documento, em que a fluência e a capacidade de síntese justificam o custo adicional.

5.3 Ameaças à Validade

Discutem-se a seguir as principais ameaças à validade deste estudo, organizadas nas categorias de construto, interna, externa e de conclusão.

Validade de construto. Refere-se a se os instrumentos medem efetivamente o que se pretende medir. A conformidade INVEST (QP2) é um construto subjetivo: como mostrado na Seção 5.2.2, a taxa de conformidade é sensível ao modelo-juiz adotado (5,9% com um modelo leve *versus* 52,5% com um modelo de raciocínio), e a concordância entre o juiz automático e o anotador humano no *corpus* AMI foi apenas leve no rótulo de conformidade plena ($\kappa = 0,13$), por divergência sobretudo na testabilidade de requisitos estéticos e na estimabilidade. Essa subjetividade é menos acentuada no domínio de requisitos de software reais, no qual a concordância entre os avaliadores foi mais que o dobro ($\kappa = 0,31$). Mitiga-se essa ameaça adotando um modelo-juiz mais capaz, instruções de avaliação explícitas e a validação por anotação humana cega em ambos os domínios, e reportando os resultados com a devida cautela quanto a esse caráter subjetivo.

Validade interna. Diz respeito a fatores que poderiam enviesar os resultados. A pontuação de conformidade INVEST e o casamento de requisitos para a cobertura (QP3), quando realizados por um único anotador, estão sujeitos a viés de julgamento; mitiga-se essa ameaça, na QP2, com a anotação de uma amostra por um segundo avaliador e a medição da concordância via *kappa* de Cohen.

Validade externa. Refere-se à capacidade de generalização. A validação no mundo real apoia-se em um único caso real (o parceiro público), e o *corpus* AMI representa um cenário específico de reuniões; assim, os resultados não se generalizam automaticamente para todos os domínios e idiomas.

Validade de conclusão. Relaciona-se à solidez das inferências. Dado o número reduzido de reuniões e o caráter de estudo de caso único da validação no mundo real, evitam-se

conclusões além do que os dados sustentam, privilegiando-se a reportagem dos resultados por categoria e a discussão de suas limitações.

Conclusão

Este trabalho investigou a automação da elicitação de requisitos de software a partir do áudio de reuniões, propondo e implementando um sistema multiagente baseado em inteligência artificial generativa. Partindo da constatação de que a elicitação manual é cara, trabalhosa e suscetível à perda de informação, o sistema decompõe o problema em quatro agentes especializados que, em conjunto, transformam uma gravação de reunião em documentação de requisitos estruturada, com dois formatos de saída selecionáveis (o padrão IEEE 830 e um formato específico de cliente). A arquitetura foi validada em um caso real, por meio de uma parceria com o Laboratório de Inovação Goiás (LIGO), e avaliada de forma quantitativa em cada um de seus estágios.

6.1 Contribuições

Em retrospecto, este trabalho entregou as seguintes contribuições:

1. **Um sistema multiagente completo e de código aberto** para a elicitação de requisitos de ponta a ponta a partir do áudio de reuniões, integrando transcrição automática, identificação de personas, geração de histórias de usuário e produção do documento de requisitos em dois formatos.
2. **Uma validação no mundo real**, sobre um projeto de software do Laboratório de Inovação Goiás (LIGO), utilizando como referência um documento de requisitos oficial produzido manualmente, cenário raramente disponível na literatura, que tende a se apoiar em dados sintéticos.
3. **Um arcabouço de avaliação quantitativa** que cobre a acurácia da transcrição, a qualidade da extração de informação e a cobertura do documento gerado frente a um documento oficial produzido manualmente, substituindo a avaliação qualitativa preliminar.
4. **Uma análise empírica da propagação de erro entre agentes**, tratando como resultado de primeira ordem o modo como erros de um estágio cascadeiam para

os artefatos seguintes, acompanhada da discussão sobre o compromisso entre custo e qualidade na seleção de modelo por agente.

Os resultados quantitativos sustentam a viabilidade da abordagem. Na transcrição (QP1), o Agente 1 obteve uma WER de 6,3% e uma CER de 2,3% em português, mais que o dobro da acurácia da alternativa local executada *offline*. Na extração de informação (QP2), o Agente 2 identificou os atores do sistema com F1 de 0,96 e gerou histórias de usuário com conformidade INVEST de 52,5% no *corpus* AMI e de 66,7% no domínio real, este último com maior concordância entre o juiz automático e o anotador humano. Na cobertura (QP3), o documento gerado recuperou 72,7% dos requisitos do documento oficial no escopo comparável, alcançando 100% nas regras de negócio e nos casos de uso; as lacunas restringiram-se a detalhes de interface ausentes do áudio. Por fim (QP4), o *pipeline* processou uma reunião real de quase 1 hora e 40 minutos em cerca de 11 minutos e ao custo de US\$ 0,72, evidenciando que a automação proposta é, ao mesmo tempo, acurada e economicamente viável.

6.2 Limitações

Os resultados deste trabalho devem ser interpretados à luz de algumas limitações. A validação no mundo real apoia-se em um **único caso real**, o que restringe a generalização das conclusões para outros domínios e organizações. Além disso, a conformidade INVEST mostrou-se um construto **subjetivo**, sensível ao modelo-juiz e com concordância apenas leve entre o juiz automático e o anotador humano, o que exige cautela na interpretação dessa métrica. Observa-se, ainda, que o documento gerado não reproduz detalhes de **interface** (caminhos de menu, títulos de tela, manipulação de telas específicas) presentes no documento oficial, por derivarem de protótipos visuais ausentes do áudio, o que representa uma limitação de modalidade do insumo, e não de extração.

Há também limitações de natureza técnica. Erros de transcrição podem **propagar-se** para os artefatos a jusante e comprometer o documento final, um modo de falha inerente a *pipelines* sequenciais; embora se tenha observado que agentes mais capazes conseguem corrigir parte desses erros pelo contexto, essa mitigação não é garantida. O sistema depende de **APIs comerciais** de transcrição e de modelos de linguagem, o que impõe custos e questões de disponibilidade e reprodutibilidade. Por fim, a avaliação concentra-se no **português** e em um domínio específico (planejamento orçamentário governamental), de modo que o desempenho em outros idiomas e domínios permanece em aberto.

6.3 Trabalhos Futuros

A partir das limitações identificadas, vislumbram-se as seguintes direções para trabalhos futuros:

- **Ampliar a validação**, processando um número maior de reuniões do *corpus* AMI e, idealmente, casos reais adicionais de outros domínios, para fortalecer a validade externa.
- **Mitigar a propagação de erro**, por exemplo, com tratamento dedicado de vocabulário técnico e de siglas de projeto, e com etapas de pós-edição ou de verificação automática entre os agentes.
- **Aumentar a cobertura** de regras de negócio e de casos de uso, investigando estratégias de *prompting* ou de pós-processamento que recuperem requisitos expressos de forma implícita.
- **Avaliar a robustez** do agente de identificação em transcrições sem rótulos de falante, medindo a dependência do sistema em relação à qualidade da segmentação por interlocutor.
- **Explorar diferentes modelos de linguagem por agente**, quantificando de forma sistemática o compromisso entre custo, latência e qualidade.
- **Incorporar revisão humana no laço** (*human-in-the-loop*), permitindo que o analista valide e ajuste os artefatos intermediários antes da geração do documento final.

Como passo de disseminação, planeja-se a redação de um artigo derivado desta monografia, a ser submetido ao *I Workshop on Software Engineering for Agentic Systems* (SE4AS 2026), realizado no âmbito do Congresso Brasileiro de Software (CBSOft 2026), cujo escopo, a engenharia de software para sistemas agênticos, é diretamente aderente à arquitetura multiagente proposta neste trabalho.

6.4 Disponibilidade de Artefatos

O código-fonte, os *scripts* de avaliação e os artefatos gerados nesta pesquisa estão publicamente disponíveis em <https://github.com/gabrielsfg/PFC1>, sob a licença MIT.

Referências Bibliográficas

- [1] ANTHROPIC. **Claude model family**. <https://www.anthropic.com>, 2025. Acessado: Junho 2026.
- [2] BOKHARI, M. U.; SIDDIQUI, S. T. **Metrics for requirements engineering and automated requirements tools**. In: *Proceedings of the 5th National Conference on Computing for Nation Development (INDIACom)*, p. 10–11, New Delhi, India, 2011.
- [3] CAI, Y.; LI, R.; LIANG, P.; SHAHIN, M.; LI, Z. **Designing LLM-based multi-agent systems for software engineering tasks: Quality attributes, design patterns and rationale**. *ACM Transactions on Software Engineering and Methodology*, 2025. arXiv:2511.08475.
- [4] CARLETTA, J. **Announcing the AMI meeting corpus**. *The ELRA Newsletter*, 11(1):3–5, January-March 2006.
- [5] COHN, M. **User Stories Applied: For Agile Software Development**. Addison-Wesley, Boston, USA, 2004.
- [6] DELGADO, A.; OTHERS. **AI-driven user story generation**. In: *Proceedings of the 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, p. 1–12. IEEE, 2024.
- [7] FELLOW.AI. **Fellow: AI meeting assistant**. <https://fellow.ai>, 2025. Acessado: Junho 2026.
- [8] GOIÁS. **Lei nº 23.042, de 29 de outubro de 2024. institui o laboratório de inovação goiás – ligo**. https://legisla.casacivil.go.gov.br/pesquisa_legislacao/109778/lei-23042, 2024. Diário Oficial do Estado de Goiás. Acessado: Julho 2026.
- [9] GROQ, INC.. **Groq API: Fast AI inference**. <https://groq.com>, 2025. Acessado: Junho 2026.
- [10] HONG, S.; ZHUGE, M.; CHEN, J.; ZHENG, X.; CHENG, Y.; ZHANG, C.; WANG, J.; WANG, Z.; YAU, S. K. S.; LIN, Z.; ZHOU, L.; RAN, C.; XIAO, L.; WU, C.;

- SCHMIDHUBER, J. **MetaGPT: Meta programming for a multi-agent collaborative framework**. In: *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*, 2024.
- [11] HORT, M.; VALLECILLOS-RUIZ, F.; MOONEN, L. **Large language models: A survey of surveys**. Preprint. <https://hal.science/hal-05341748>, 2025.
- [12] IEEE. **IEEE Recommended Practice for Software Requirements Specifications**. Institute of Electrical and Electronics Engineers, New York, USA, 1998.
- [13] JAMIE. **Jamie: AI note-taking assistant**. <https://www.meetjamie.ai>, 2025. Acessado: Junho 2026.
- [14] JIN, D.; JIN, Z.; CHEN, X.; WANG, C. **MARE: Multi-agents collaboration framework for requirements engineering**. *arXiv preprint arXiv:2405.03256*, 2024.
- [15] KASAKOWSKIJ, T.; HAAKE, J. M. **T³ Talk2Text: A model for near real-time voice transcription in virtual group meetings**. *Discover Education*, 4:146, 2025.
- [16] LUCASSEN, G.; DALPIAZ, F.; VAN DER WERF, J. M. E. M.; BRINKKEMPER, S. **Improving agile requirements: The quality user story framework and tool**. *Requirements Engineering*, 21(3):383–403, 2016.
- [17] LUO, Q.; YE, Y.; LIANG, S.; ZHANG, Z.; QIN, Y.; LU, Y.; WU, Y.; CONG, X.; LIN, Y.; ZHANG, Y.; CHE, X.; LIU, Z.; SUN, M. **RepoAgent: An LLM-powered open-source framework for repository-level code documentation generation**. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, p. 436–464. Association for Computational Linguistics, 2024.
- [18] MOTGER, Q.; CATOT, C.; FRANCH, X. **ShaRE your data! characterizing datasets for LLM-based requirements engineering**. In: *arXiv preprint arXiv:2510.18787*, 2025.
- [19] NAIMI, L.; BOUZIANE, E. M.; JAKIMI, A.; SAADANE, R.; CHEHRI, A. **Automating software documentation: Employing LLMs for precise use case description**. In: *Procedia Computer Science*, volume 246, p. 1346–1354. Elsevier, 2024.
- [20] OPENNMT. **CTranslate2: Fast inference engine for transformer models**. <https://github.com/OpenNMT/CTranslate2>, 2024.
- [21] POHL, K. **Requirements Engineering: Fundamentals, Principles, and Techniques**. Springer, Berlin, Heidelberg, 2010.

- [22] QIAN, C.; LIU, W.; LIU, H.; CHEN, N.; DANG, Y.; LI, J.; YANG, C.; CHEN, W.; SU, Y.; CONG, X.; XU, J.; LI, D.; LIU, Z.; SUN, M. **ChatDev: Communicative agents for software development**. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*. Association for Computational Linguistics, 2024.
- [23] RADFORD, A.; KIM, J. W.; XU, T.; OTHERS. **Robust speech recognition via large-scale weak supervision**. *arXiv preprint arXiv:2212.04356*, 2022.
- [24] REN, S.; NAKAGAWA, H.; TSUCHIYA, T. **LLM-based class diagram derivation from user stories with chain-of-thought promptings**. In: *Proceedings of the 2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*, p. 1376–1381. IEEE, 2024.
- [25] SAMI, M. A.; WASEEM, M.; RASHEED, Z.; SAARI, M.; SYSTÄ, K.; ABRAHAMSSON, P. **Experimenting with multi-agent software development: Towards a unified platform**, 2024.
- [26] SANWAL, M.; DEVA, I. **An autonomous multi-agent LLM framework for agile software development**. *International Journal of Trend in Scientific Research and Development (IJTSRD)*, 8(5):892–898, 2024. <https://www.ijtsrd.com/papers/ijtsrd70467.pdf>.
- [27] SOMMERVILLE, I. **Software Engineering**. Pearson, Boston, USA, 10 edition, 2015.
- [28] SU, Y.; WAN, C.; SETHI, U.; LU, S.; MUSUVATHI, M.; NATH, S. **HotGPT: How to make software documentation more useful with a large language model?** In: *Proceedings of the 19th Workshop on Hot Topics in Operating Systems (HotOS '23)*, p. 87–93, Providence, RI, USA, 2023. ACM.
- [29] VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, L.; POLOSUKHIN, I. **Attention is all you need**. In: *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [30] WAKE, B. **INVEST in good stories, and SMART tasks**. <https://xp123.com/articles/invest-in-good-stories-and-smart-tasks/>, 2003. Acessado: Junho 2026.
- [31] WOOLDRIDGE, M. **An Introduction to MultiAgent Systems**. John Wiley & Sons, Chichester, UK, 2 edition, 2009.
- [32] XU, X.; DOU, Y.; QIAN, L.; ZHANG, Z.; MA, Y.; TAN, Y. **A requirement quality assessment method based on user stories**. *Electronics*, 12(9):2155, 2023.

Prompts do Agente de Identificação

Este apêndice reproduz os *prompts* utilizados pelo Agente 2 (identificação de personas e geração de histórias de usuário), tal como definidos em `agente-identificacao/config/prompts.py`. Os marcadores entre chaves — `{transcription}` e `{personas_json}` — são substituídos em tempo de execução, respectivamente, pela transcrição da reunião e pelo JSON de personas produzido na primeira chamada.

A.1 Mensagem de Sistema

Você é um especialista em engenharia de requisitos e análise de reuniões de elicitação. Sua tarefa é analisar a transcrição de uma reunião e separar claramente DOIS conceitos:

1. **PERSONAS / ATORES DO SISTEMA:** os perfis de usuário que vão USAR o software discutido (ex.: Administrador, Operador, Cliente, Gestor, Usuário do órgão). Você os infere a partir das funcionalidades descritas, MESMO que essas pessoas não estejam na reunião.
2. **PARTICIPANTES DA REUNIÃO:** as pessoas que efetivamente falaram (equipe do projeto: analistas, desenvolvedores, gerente de projeto, prototipador). Servem apenas como metadado de rastreabilidade e NÃO são as personas do sistema -- a menos que sejam, claramente, também usuárias do software.

Os requisitos e casos de uso derivados depois serão escritos do ponto de vista das PERSONAS (atores do sistema). Seja preciso e baseie-se apenas na transcrição.

A.2 Prompt de Identificação de Personas

Analise a transcrição de reunião abaixo e produza a análise de personas do SISTEMA que está sendo discutido.

TRANSCRIÇÃO:

```
{transcription}
```

IMPORTANTE -- distinga DOIS conceitos:

- "personas": os ATORES / USUÁRIOS DO SISTEMA (perfis que vão usar o software), inferidos das funcionalidades discutidas (ex.: "Operador de Orçamento", "Gestor", "Administrador"). Mesmo que essas pessoas não estejam presentes na reunião. NÃO use os nomes dos participantes da reunião como personas (a não ser que sejam, claramente, também usuários do sistema). Identifique de 2 a 6 atores distintos, consolidando perfis semelhantes.
- "participantes_reuniao": as pessoas que efetivamente FALARAM na reunião (equipe do projeto). Apenas metadado.

Separe também, de forma RESUMIDA, as DISCUSSÕES DE GESTÃO DE PROJETO (cronograma, EAP, prazos, alocação de equipe, reuniões de validação, priorização de escopo, mapeamento de código, transferência de conhecimento) no campo "gestao_projeto" -- essas discussões NÃO são requisitos do software, mas devem ser registradas à parte.

Retorne em formato JSON:

```
{
  "personas": [
    {
      "id": "persona_1",
      "nome": "Nome do ATOR DO SISTEMA (ex.: Operador de Orçamento)",
      "papel": "Perfil/função de uso no sistema",
      "caracteristicas": ["característica 1", "característica 2"],
      "contexto": "Como esse ator usa o sistema e o que precisa",
      "trechos_relevantes": ["Trecho que evidencia esse ator"]
    }
  ],
  "participantes_reuniao": [
    { "nome": "Nome de quem falou",
      "papel_na_reuniao": "ex.: analista, desenvolvedor, gerente" }
  ],
  "resumo_conversa": "Resumo do sistema/funcionalidade discutidos",
  "tipo_interacao": "ex.: reunião de levantamento de requisitos",
  "gestao_projeto": {
    "resumo": "1 a 3 frases sobre decisões de gestão (não requisitos)",
    "pontos": ["ponto principal 1", "ponto principal 2"]
  }
}
```

Seja preciso e baseie-se apenas na transcrição.

Retorne APENAS o JSON, sem texto adicional antes ou depois.

A.3 Prompt de Geração de Histórias de Usuário

Com base nas PERSONAS DO SISTEMA (atores) abaixo, crie histórias de usuário (user stories) seguindo os critérios INVEST (Independente, Negociável, Valiosa, Estimável, Pequena e Testável).

PERSONAS (ATORES DO SISTEMA):

```
{personas_json}
```

Para cada persona (ator do sistema), crie histórias no formato:

"Como [ator do sistema], eu quero [objetivo no sistema] para [benefício/razão]"

REGRAS IMPORTANTES:

- Escreva do ponto de vista do ATOR DO SISTEMA (use o campo "personas", NÃO os "participantes_reuniao").
- Inclua APENAS necessidades de COMPORTAMENTO DO SOFTWARE. NÃO crie histórias sobre gestão de projeto (cronograma, EAP, prazos, alocação de equipe, reuniões, mapeamento de código, transferência de conhecimento).
- Consolidar histórias redundantes; foque no que o sistema deve fazer para cada ator.

Retorne em formato JSON:

```
{
  "user_stories": [
    {
      "persona_id": "persona_1",
      "persona_nome": "Nome da persona",
      "historias": [
        {
          "id": "story_1",
          "historia": "Como [persona], eu quero [objetivo] para [...]",
          "prioridade": "alta|média|baixa",
          "contexto": "Contexto adicional da história",
          "trecho_base": "Trecho da transcrição que originou a história"
        }
      ]
    }
  ]
}
```

Retorne APENAS o JSON, sem texto adicional.

Prompts do Agente de Identificação

Este apêndice reproduz os *prompts* utilizados pelo Agente 2 (identificação de personas e geração de histórias de usuário), tal como definidos em `agente-identificacao/config/prompts.py`. Os marcadores entre chaves, `{transcription}` e `{personas_json}`, são substituídos em tempo de execução, respectivamente, pela transcrição da reunião e pelo JSON de personas produzido na primeira chamada.

B.1 Mensagem de Sistema

Você é um especialista em engenharia de requisitos e análise de reuniões de elicitação. Sua tarefa é analisar a transcrição de uma reunião e separar claramente DOIS conceitos:

1. **PERSONAS / ATORES DO SISTEMA:** os perfis de usuário que vão USAR o software discutido (ex.: Administrador, Operador, Cliente, Gestor, Usuário do órgão). Você os infere a partir das funcionalidades descritas, MESMO que essas pessoas não estejam na reunião.
2. **PARTICIPANTES DA REUNIÃO:** as pessoas que efetivamente falaram (equipe do projeto: analistas, desenvolvedores, gerente de projeto, prototipador). Servem apenas como metadado de rastreabilidade e NÃO são as personas do sistema -- a menos que sejam, claramente, também usuárias do software.

Os requisitos e casos de uso derivados depois serão escritos do ponto de vista das PERSONAS (atores do sistema). Seja preciso e baseie-se apenas na transcrição.

B.2 Prompt de Identificação de Personas

Analise a transcrição de reunião abaixo e produza a análise de personas do SISTEMA que está sendo discutido.

TRANSCRIÇÃO:

```
{transcription}
```

IMPORTANTE -- distinga DOIS conceitos:

- "personas": os ATORES / USUÁRIOS DO SISTEMA (perfis que vão usar o software), inferidos das funcionalidades discutidas (ex.: "Operador de Orçamento", "Gestor", "Administrador"). Mesmo que essas pessoas não estejam presentes na reunião. NÃO use os nomes dos participantes da reunião como personas (a não ser que sejam, claramente, também usuários do sistema). Identifique de 2 a 6 atores distintos, consolidando perfis semelhantes.
- "participantes_reuniao": as pessoas que efetivamente FALARAM na reunião (equipe do projeto). Apenas metadado.

Separe também, de forma RESUMIDA, as DISCUSSÕES DE GESTÃO DE PROJETO (cronograma, EAP, prazos, alocação de equipe, reuniões de validação, priorização de escopo, mapeamento de código, transferência de conhecimento) no campo "gestao_projeto" -- essas discussões NÃO são requisitos do software, mas devem ser registradas à parte.

Retorne em formato JSON:

```
{
  "personas": [
    {
      "id": "persona_1",
      "nome": "Nome do ATOR DO SISTEMA (ex.: Operador de Orçamento)",
      "papel": "Perfil/função de uso no sistema",
      "caracteristicas": ["característica 1", "característica 2"],
      "contexto": "Como esse ator usa o sistema e o que precisa",
      "trechos_relevantes": ["Trecho que evidencia esse ator"]
    }
  ],
  "participantes_reuniao": [
    { "nome": "Nome de quem falou",
      "papel_na_reuniao": "ex.: analista, desenvolvedor, gerente" }
  ],
  "resumo_conversa": "Resumo do sistema/funcionalidade discutidos",
  "tipo_interacao": "ex.: reunião de levantamento de requisitos",
  "gestao_projeto": {
    "resumo": "1 a 3 frases sobre decisões de gestão (não requisitos)",
    "pontos": ["ponto principal 1", "ponto principal 2"]
  }
}
```

Seja preciso e baseie-se apenas na transcrição.

Retorne APENAS o JSON, sem texto adicional antes ou depois.

B.3 Prompt de Geração de Histórias de Usuário

Com base nas PERSONAS DO SISTEMA (atores) abaixo, crie histórias de usuário (user stories) seguindo os critérios INVEST (Independente, Negociável, Valiosa, Estimável, Pequena e Testável).

PERSONAS (ATORES DO SISTEMA):

```
{personas_json}
```

Para cada persona (ator do sistema), crie histórias no formato:

"Como [ator do sistema], eu quero [objetivo no sistema] para [benefício/razão]"

REGRAS IMPORTANTES:

- Escreva do ponto de vista do ATOR DO SISTEMA (use o campo "personas", NÃO os "participantes_reuniao").
- Inclua APENAS necessidades de COMPORTAMENTO DO SOFTWARE. NÃO crie histórias sobre gestão de projeto (cronograma, EAP, prazos, alocação de equipe, reuniões, mapeamento de código, transferência de conhecimento).
- Consolidar histórias redundantes; foque no que o sistema deve fazer para cada ator.

Retorne em formato JSON:

```
{
  "user_stories": [
    {
      "persona_id": "persona_1",
      "persona_nome": "Nome da persona",
      "historias": [
        {
          "id": "story_1",
          "historia": "Como [persona], eu quero [objetivo] para [...]",
          "prioridade": "alta|média|baixa",
          "contexto": "Contexto adicional da história",
          "trecho_base": "Trecho da transcrição que originou a história"
        }
      ]
    }
  ]
}
```

Retorne APENAS o JSON, sem texto adicional.

Contrato de Dados entre Agentes

A comunicação entre os agentes do *pipeline* ocorre por meio de arquivos. O artefato central é o documento JSON produzido pelo Agente 2, que serve de **contrato** entre a identificação de personas (Agente 2) e a geração do documento de requisitos (Agente 3). Este apêndice apresenta a estrutura desse documento. Os campos detalhados de *personas* e de *user_stories* seguem os esquemas reproduzidos no Apêndice B.

```
{
  "metadata": {
    "arquivo_origem": "nome do arquivo de transcrição de entrada",
    "data_processamento": "carimbo de data/hora do processamento",
    "modelo_usado": "modelo de linguagem utilizado (ex.: claude-haiku-4-5)"
  },
  "analise_personas": {
    "personas": [ /* atores do sistema (ver Apêndice A) */ ],
    "participantes_reuniao": [ /* quem falou na reunião (metadado) */ ],
    "resumo_conversa": "resumo do sistema/funcionalidade discutidos",
    "tipo_interacao": "ex.: reunião de levantamento de requisitos",
    "gestao_projeto": { /* decisões de gestão (não são requisitos) */ }
  },
  "historias_usuario": {
    "user_stories": [ /* histórias por persona, no formato INVEST */ ]
  }
}
```

A separação explícita entre *personas* (atores do sistema) e *participantes_reuniao* (quem efetivamente falou na reunião) é uma decisão de projeto deliberada: ela evita que participantes da reunião sejam tratados, por engano, como atores do software, um modo de falha observado em versões anteriores do sistema. De modo análogo, o campo *gestao_projeto* isola as discussões de gestão (cronograma, prazos, alocação de equipe), de forma que elas não contaminem os requisitos funcionais derivados a jusante.